11
OFFENSIVE FORENSICS



Forensics folks are often the people called in after a breach, or to determine if an “incident” has taken place at all. They typically want a snapshot of the affected machine’s RAM in order to capture cryptographic keys or other information that resides only in memory. Lucky for them, a team of talented developers has created an entire Python framework called *Volatility* that’s suitable for this task and is billed as an advanced memory forensics framework. Incident responders, forensic examiners, and malware analysts can use Volatility for a variety of other tasks as well, including inspecting kernel objects, examining and dumping processes, and so on.

Although Volatility is software for the defensive side, any sufficiently powerful tool can be used for offense or defense. We will use Volatility to perform reconnaissance on a target user and write our own offensive plug-ins to search for weakly defended processes running on a virtual machine (VM).

Suppose you infiltrate a machine and discover that the user employs a VM for sensitive work. Chances are good that the user has also made a snapshot of the VM as a safety net in case anything goes wrong with it. We will use the Volatility memory analysis framework to analyze the snapshot to find out how the VM is used and what processes were running. We'll also investigate possible vulnerabilities we can leverage for further exploitation.

Let's get started!

Installation

Volatility has been around for several years and has just undergone a complete rewrite. Not only is the code base now founded on Python 3, but the entire framework has been refactored so that the components are independent; all state required to run a plug-in is self-contained.

Let's create a virtual environment just for our work with Volatility. For this example, we are using Python 3 on a Windows machine in a PowerShell terminal. If you are also working from a Windows machine, make sure you have `git` installed. You can download it at *https://git-scm.com/downloads/*.

```
1 PS> python3 -m venv vol3
   PS> vol3/Scripts/Activate.ps1
   PS> cd vol3/
2 PS> git clone
https://github.com/volatilityfoundation/volatility3.git
   PS> cd volatility3/
   PS> python setup.py  install
3 PS> pip install pycryptodome
```

First, we create a new virtual environment called `vol3` and activate it 1. Next, we move into the virtual environment directory and clone the Volatility 3 GitHub repo 2, install it into the virtual environment, and finally install `pycryptodome`3, which we'll need later.

To see the plug-ins Volatility offers, as well as a list of options, use the following command on Windows:

```
PS> vol --help
```

On Linux or Mac, use the Python executable from the virtual environment, as follows:

```
$> python vol.py --help
```

In this chapter, we'll use Volatility from the command line, but there are various ways you might encounter the framework. For example, see the Volumetric project from Volatility, a free web-based GUI for volatility (*https://github.com/volatilityfoundation/volumetric/*). You can dig into code examples in the Volumetric project to see how you can use Volatility in your own programs. Additionally, you can use the `volshell` interface, which provides you with access to the Volatility framework and works as a normal interactive Python shell.

In the examples that follow, we'll use the Volatility command line. To save space, the output has been edited to show only the output discussed, so be aware that your output will have more lines and columns.

Now let's delve into some code and have a look inside the framework:

```
PS> cd volatility/framework/plugins/windows/
PS> ls
_init__.py    driverscan.py memmap.py       psscan.py
vadinfo.py
bigpools.py   filescan.py   modscan.py      pstree.py
vadyarascan.py
cachedump.py  handles.py    modules.py      registry/
verinfo.py
callbacks.py  hashdump.py   mutantscan.py   ssdt.py
virtmap.py
cmdline.py    info.py       netscan.py      strings.py
dlllist.py    lsadump.py    poolscanner.py  svcscan.py
driverirp.py  malfind.py    pslist.py       symlinkscan.py
```

This listing shows the Python files inside the Windows *plugin* directory. We highly encourage you to spend some time looking at the code in these files. You'll see a recurring pattern that forms the structure of a Volatility plug-in. This will help you understand the framework, but more importantly, it will give you a picture of a defender's mindset and intentions. By knowing what defenders are capable of and how they accomplish their objectives, you will make yourself into a more capable hacker and better understand how to protect yourself from detection.

Now that we have the analysis framework ready, we need some memory images to analyze. The easiest way to get one is to take a snapshot of your own Windows 10 virtual machine.

First, power up your Windows VM and start a few processes (for instance, the notepad, the calculator, and a browser); we'll examine the memory and track how these processes started. Then, take your snapshot using your hypervisor of choice. In the directory where your hypervisor stores your VMs, you'll see your new snapshot file with a name that ends with *.vmem* or *.mem*. Let's start doing some recon!

Note that you can also find many memory images online. One image we'll look at in this chapter is provided by PassMark Software at *https://www.osforensics.com/tools/volatility-workbench.html/*. The Volatility Foundation site also has several images to play with at *https://github.com/volatilityfoundation/volatility/wiki/Memory-Samples/*.

General Reconnaissance

Let's get an overview of the machine we're analyzing. The `windows.info` plug-in shows the operating system and kernel information of the memory sample:

```
1 PS>vol -f WinDev2007Eval-Snapshot4.vmem windows.info
  Volatility 3 Framework 1.2.0-beta.1
  Progress:   33.01               Scanning primary2 using
  PdbSignatureScanner
  Variable        Value
```

```
Kernel Base     0xf80067a18000
DTB             0x1aa000
primary 0       WindowsIntel32e
memory_layer    1 FileLayer
KdVersionBlock  0xf800686272f0
Major/Minor     15.19041
MachineType     34404
KeNumberProcessors      1
SystemTime      2020-09-04 00:53:46
NtProductType   NtProductWinNt
NtMajorVersion  10
NtMinorVersion  0
PE MajorOperatingSystemVersion  10
PE MinorOperatingSystemVersion  0
PE Machine      34404
```

We specify the snapshot filename with the `-f` switch and the Windows plug-in to use, `windows.info`1. Volatility reads and analyzes the memory file and outputs general information about this Windows machine. We can see that we're dealing with a Windows 10.0 VM and that it has a single processor and a single memory layer.

You might find it educational to try several plug-ins on the memory image file while reviewing the plug-in code. Spending time reading code and seeing the corresponding output will show you how the code is supposed to work as well as the general mindset of the defenders.

Next, with the `registry.printkey` plug-in, we can print the values of a key in the registry. There is a wealth of information in the registry, and Volatility provides a way to find any value we wish. Here, we look for installed services. The key */ControlSet001/Services* shows the Service Control Manager database, which lists all the installed services:

```
PS>vol -f WinDev2007Eval-7d959ee5.vmem
windows.registry.printkey --key 'ControlSet001\Services'
Volatility 3 Framework 1.2.0-beta.1
Progress:   33.01               Scanning primary2 using
PdbSignatureScanner
... Key                                         Name    Data
Volatile
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services .NET CLR Data
```

```
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services Appinfo
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services applockerfltr
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services
AtomicAlarmClock   False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services Beep
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services fastfat
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services
MozillaMaintenance False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services NTDS
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services Ntfs
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services
ShellHWDetection   False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services SQLWriter
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services Tcpip
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services Tcpip6
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services terminpt
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services W32Time
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services WaaSMedicSvc
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services WacomPen
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services Winsock
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services WinSock2
False
\REGISTRY\MACHINE\SYSTEM\ControlSet001\Services WINUSB
False
```

This output shows a list of installed services on the machine (abbreviated for space).

User Reconnaissance

Now let's do some recon on the user of the VM. The `cmdline` plug-in lists the command line arguments for each process as they were running at the time the snapshot was made. These processes give us a hint as to the user's behavior and intent.

```
PS>vol -f WinDev2007Eval-7d959ee5.vmem windows.cmdline
Volatility 3 Framework 1.2.0-beta.1
Progress:   33.01               Scanning primary2 using
PdbSignatureScanner
PID     Process Args

72      Registry        Required memory at 0x20 is not valid
(process exited?)
340     smss.exe        Required memory at 0xa5f1873020 is
inaccessible (swapped)
564     lsass.exe       C:\Windows\system32\lsass.exe
624     winlogon.exe    winlogon.exe
2160    MsMpEng.exe     "C:\ProgramData\Microsoft\Windows
Defender\platform\4.18.2008.9-0\MsMpEng.exe"
4732    explorer.exe    C:\Windows\Explorer.EXE
4848    svchost.exe     C:\Windows\system32\svchost.exe -k
ClipboardSvcGroup -p
4920    dllhost.exe     C:\Windows\system32\DllHost.exe
/Processid:{AB8902B4-09CA-4BB6-B78D-A8F59079A8D5}
5084    StartMenuExper
"C:\Windows\SystemApps\Microsoft.Windows. . ."
5388    MicrosoftEdge.
"C:\Windows\SystemApps\Microsoft.MicrosoftEdge_. . ."
6452    OneDrive.exe
"C:\Users\Administrator\AppData\Local\Microsoft\OneDrive\OneD
rive.exe" /background
6484    FreeDesktopClo  "C:\Program Files\Free Desktop
Clock\FreeDesktopClock.exe"
7092    cmd.exe         "C:\Windows\system32\cmd.exe" 1
3312    notepad.exe     notepad 2
3824    powershell.exe
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
6448    Calculator.exe  "C:\Program
Files\WindowsApps\Microsoft.WindowsCalculator_. . ."
6684    firefox.exe     "C:\Program Files (x86)\Mozilla
Firefox\firefox.exe"
6432    PowerToys.exe   "C:\Program
Files\PowerToys\PowerToys.exe"
7124    nc64.exe        Required memory at 0x2d7020 is
inaccessible (swapped)
```

```
3324    smartscreen.ex  C:\Windows\System32\smartscreen.exe -
Embedding
4768    ipconfig.exe    Required memory at 0x840308e020 is
not valid (process exited?)
```

The list shows the process ID, process name, and command line with arguments that started the process. You can see that most processes were started by the system itself, most likely at boot time. The `cmd.exe`1 and `notepad.exe`2 processes are typical processes a user would start.

Let's investigate the running processes a little bit deeper with the `pslist` plug-in, which lists the processes that were running at the time of the snapshot.

```
PS>vol -f WinDev2007Eval-7d959ee5.vmem windows.pslist
Volatility 3 Framework 1.2.0-beta.1
Progress:   33.01               Scanning primary2 using
PdbSignatureScanner
PID     PPID    ImageFileName   Offset(V)   Threads Handles
SessionId   Wow64

4       0       System          0xa50bb3e6d040  129     -
N/A     False
72      4       Registry        0xa50bb3fbd080  4       -
N/A     False
6452    4732    OneDrive.exe    0xa50bb4d62080  25      -
1       True
6484    4732    FreeDesktopClo  0xa50bbb847300  1       -
1       False
6212    556     SgrmBroker.exe  0xa50bbb832080  6       -
0       False
1636    556     svchost.exe     0xa50bbadbe340  8       -
0       False
7092    4732    cmd.exe         0xa50bbbc4d080  1       -
1       False
3312    7092    notepad.exe     0xa50bbb69a080  3       -
1       False
3824    4732    powershell.exe  0xa50bbb92d080  11      -
1       False
6448    704     Calculator.exe  0xa50bb4d0d0c0  21      -
1       False
4036    6684    firefox.exe     0xa50bbb178080  0       -
1       True
6432    4732    PowerToys.exe   0xa50bb4d5a2c0  14      -
```

```
1       False
4052    4700    PowerLauncher.  0xa50bb7fd3080  16      -
1       False
5340    6432    Microsoft.Powe  0xa50bb736f080  15      -
1       False
8564    4732    python-3.8.6-a  0xa50bb7bc2080  1       -
1       True
7124    7092    nc64.exe        0xa50bbab89080  1       -
1       False
3324    704     smartscreen.ex  0xa50bb4d6a080  7       -
1       False
7364    4732    cmd.exe         0xa50bbd8a8080  1       -
1       False
8916    2136    cmd.exe         0xa50bb78d9080  0       -
0       False
4768    8916    ipconfig.exe    0xa50bba7bd080  0       -
0       False
```

Here we see the actual processes and their memory offsets. Some columns have been omitted for space. Several interesting processes are listed, including the `cmd` and `notepad` processes we saw in the output from the `cmdline` plug-in.

It would be nice to see the processes as a hierarchy, so we can tell what process started other processes. For that, we'll use the `pstree` plug-in:

```
PS>vol -f WinDev2007Eval-7d959ee5.vmem windows.pstree

Volatility 3 Framework 1.2.0-beta.1
Progress:   33.01               Scanning primary2 using
PdbSignatureScanner
PID     PPID    ImageFileName   Offset(V)    Threads Handles
SessionId Wow64
4         0     System          0xa50bba7bd080  129     N/A
False
* 556    492     services.exe    0xa50bba7bd080    8       0
False
** 2176  556     wlms.exe        0xa50bba7bd080    2       0
False
** 1796  556     svchost.exe     0xa50bba7bd080   13       0
False
** 776   556     svchost.exe     0xa50bba7bd080   15       0
False
** 8     556     svchost.exe     0xa50bba7bd080   18       0
```

```
False
*** 4556   8      ctfmon.exe      0xa50bba7bd080   10        1
False
*** 5388 704      MicrosoftEdge.  0xa50bba7bd080   35        1
False
*** 6448 704      Calculator.exe  0xa50bba7bd080   21        1
False
*** 3324 704      smartscreen.ex  0xa50bba7bd080    7        1
False
** 2136  556      vmtoolsd.exe    0xa50bba7bd080   11        0
False
*** 8916 2136     cmd.exe         0xa50bba7bd080   0         0
False
**** 4768 8916    ipconfig.exe    0xa50bba7bd080   0         0
False

* 4704           624    userinit.exe      0xa50bba7bd080  0
1 False
** 4732         4704     explorer.exe     0xa50bba7bd080  92
1 False
*** 6432        4732     PowerToys.exe    0xa50bba7bd080  14
1 False
**** 5340       6432      Microsoft.Powe  0xa50bba7bd080  15
1 False
*** 7364        4732     cmd.exe          0xa50bba7bd080  1
- False
**** 2464       7364      conhost.exe     0xa50bba7bd080  4
1 False
*** 7092        4732     cmd.exe          0xa50bba7bd080  1
- False
**** 3312       7092      notepad.exe     0xa50bba7bd080  3
1 False
**** 7124       7092      nc64.exe        0xa50bba7bd080  1
1 False
*** 8564        4732     python-3.8.6-a   0xa50bba7bd080  1
1 True
**** 1036       8564      python-3.8.6-a  0xa50bba7bd080  5
1 True
```

Now we get a clearer picture. The asterisk in each row indicates the parent-child relationship of the process. For example, the `userinit` process (PID 4704) spawned the `explorer.exe` process. Likewise, the `explorer.exe` process (PID 4732) started the `cmd.exe` process (PID 7092). From that process, the user started `notepad.exe` and another process called `nc64.exe`.

Now let's check for passwords with the `hashdump` plug-in:

```
PS> vol -f WinDev2007Eval-7d959ee5.vmem windows.hashdump
Volatility 3 Framework 1.2.0-beta.1
Progress:   33.01               Scanning primary2 using
PdbSignatureScanner
User                rid     lmhash
nthash

Administrator       500     aad3bXXXXXXaad3bXXXXXX
fc6eb57eXXXXXXXXXXX657878
Guest               501     aad3bXXXXXXaad3bXXXXXX
1d6cfe0dXXXXXXXXXXXc089c0
DefaultAccount      503     aad3bXXXXXXaad3bXXXXXX
1d6cfe0dXXXXXXXXXXXc089c0
WDAGUtilityAccount 504      aad3bXXXXXXaad3bXXXXXX
ed66436aXXXXXXXXXXX1bb50f
User               1001     aad3bXXXXXXaad3bXXXXXX
31d6cfe0XXXXXXXXXXXc089c0
tim                1002     aad3bXXXXXXaad3bXXXXXX
afc6eb57XXXXXXXXXXX657878
admin              1003     aad3bXXXXXXaad3bXXXXXX
afc6eb57XXXXXXXXXXX657878
```

The output shows the account usernames and the LM and NT hashes of their passwords. Recovering the password hashes on a Windows machine after penetration is a common goal of attackers. These hashes can be cracked offline in an attempt to recover the target's password, or they can be used in a pass-the-hash attack to gain access to other network resources. Whether the target is a paranoid user who performs high-risk operations only on a VM or is an enterprise attempting to contain some of its users' activities to VMs, looking through the VMs or snapshots on the system is a perfect time for attempting to recover these hashes after you've gained access to the host hardware.

Volatility makes this recovery process extremely easy.

We've obfuscated the hashes in our output. You can use your own output as input to a hash-cracking tool to find your way into the VM. There are several online hash-cracking sites; alternatively, you can use John the Ripper on your Kali machine.

Vulnerability Reconnaissance

Now let's use Volatility to discover whether the target VM has vulnerabilities we may be able to exploit. The `malfind` plug-in checks for process memory ranges that potentially contain injected code. *Potential* is the key word here—the plug-in is looking for regions of memory that have permissions to read, write, and execute. It is worthwhile to investigate these processes since they may enable us to leverage some malware that is already available. Alternatively, we may be able to overwrite those regions with our own malware.

```
PS>vol -f WinDev2007Eval-7d959ee5.vmem windows.malfind
Volatility 3 Framework 1.2.0-beta.1
Progress:   33.01               Scanning primary2 using
PdbSignatureScanner
PID  Process         Start VPN     End VPN       Tag
Protection       CommitCharge

1336 timeserv.exe    0x660000      0x660fff      VadS
PAGE_EXECUTE_READWRITE  1
2160 MsMpEng.exe     0x16301690000 0x1630179cfff VadS
PAGE_EXECUTE_READWRITE  269
2160 MsMpEng.exe     0x16303090000 0x1630318ffff VadS
PAGE_EXECUTE_READWRITE  256
2160 MsMpEng.exe     0x16304a00000 0x16304bfffff VadS
PAGE_EXECUTE_READWRITE  512
6484 FreeDesktopClo  0x2320000     0x2320fff     VadS
PAGE_EXECUTE_READWRITE  1
5340 Microsoft.Powe  0x2c2502c0000 0x2c2502cffff VadS
PAGE_EXECUTE_READWRITE  15
```

We've encountered a couple of potential problems. The `timeserv.exe` process (PID 1336) is part of the freeware known as `FreeDesktopClock` (PID 6484). These processes are not necessarily a problem as long as they're installed under *C:\Program Files*. Otherwise, the process may be malware masquerading as a clock.

Using a search engine, you will find that the process `MsMpEng.exe` (PID 2160) is an anti-malware service. Even though these processes contain writable and executable memory regions, they don't appear to be dangerous. Perhaps we could make these processes

dangerous by writing shellcode into those memory regions, so it's worth taking note of them.

The `netscan` plug-in provides a list of all the network connections the machine had at the time of the snapshot, as shown next. Anything that looks suspicious we may be able to leverage in an attack.

```
PS>vol -f WinDev2007Eval-7d959ee5.vmem windows.netscan
Volatility 3 Framework 1.2.0-beta.1
Progress:   33.01               Scanning primary2 using
PdbSignatureScanner
Offset         Proto LocalAddr  LocalPort ForeignAdd
ForeignPort State     PID  Owner

0xa50bb7a13d90 TCPv4 0.0.0.0         4444 0.0.0.0 0
LISTENING   7124 nc64.exe 1
0xa50bb9f4c310 TCPv4 0.0.0.0         7680 0.0.0.0 0
LISTENING   1776 svchost.exe
0xa50bb9f615c0 TCPv4 0.0.0.0        49664 0.0.0.0 0
LISTENING   564  lsass.exe
0xa50bb9f62190 TCPv4 0.0.0.0        49665 0.0.0.0 0
LISTENING   492  wininit.exe
0xa50bbaa80b20 TCPv4 192.168.28.128  50948 23.40.62.19    80
CLOSED 2
w0xa50bbabd2010 TCPv4 192.168.28.128 50954 23.193.33.57   443
CLOSED
0xa50bbad8d010 TCPv4 192.168.28.128  50953 99.84.222.93   443
CLOSED
0xa50bbaef3010 TCPv4 192.168.28.128  50959 23.193.33.57   443
CLOSED
0xa50bbaff7010 TCPv4 192.168.28.128  50950 52.179.224.121 443
CLOSED
0xa50bbbd240a0 TCPv4 192.168.28.128    139   0.0.0.0 0
LISTENING
```

We see some connections from the local machine (192.168.28.128), apparently to a couple of web servers 2; these connections are now closed. More important are the connections marked `LISTENING`. The ones that are owned by recognizable Windows processes (`svchost`, `lsass`, `wininit`) may be okay, but the `nc64.exe` process is unknown 1. It is listening on port 4444, and it's

well worth taking a deeper look by using our `netcat` substitute from Chapter 2 to probe that port.

The volshell Interface

In addition to the command line interface, you can use Volatility in a custom Python shell with the `volshell` command. This gives you all the power of Volatility as well as a full Python shell. Here is an example of using the `pslist` plug-in on a Windows image using `volshell`:

```
PS> volshell -w -f WinDev2007Eval-7d959ee5.vmem 1
>>> from volatility.plugins.windows import pslist 2
>>> dpo(pslist.PsList, primary=self.current_layer,
nt_symbols=self.config['nt_symbols']) 3
PID     PPID    ImageFileName   Offset(V)   Threads Handles
SessionId   Wow64

4       0       System          0xa50bb3e6d040  129     -
N/A     False
72      4       Registry        0xa50bb3fbd080  4       -
N/A     False
6452    4732    OneDrive.exe    0xa50bb4d62080  25      -
1       True
6484    4732    FreeDesktopClo  0xa50bbb847300  1       -
1       False
...
```

In this brief example, we used the `-w` switch to tell Volatility that we're analyzing a Windows image and the `-f` switch to specify the image itself 1. Once we're in the `volshell` interface, we use it just like a normal Python shell. That is, you can import packages or write functions as you normally would, but now you also have Volatility embedded in the shell. We import the `pslist` plug-in 2 and display output (the `dpo` function) from the plug-in 3.

You can find more information on using `volshell` by entering **`volshell --help`**.

Custom Volatility Plug-Ins

We've just seen how we can use the Volatility plug-ins to analyze a VM snapshot for existing vulnerabilities, profile the user by checking the commands and processes in use, and dump the password hashes. But since you can write your own custom plug-ins, only your imagination limits what you can do with Volatility. If you need additional information based on clues found from the standard plug-ins, you can make a plug-in of your own.

The Volatility team has made it easy to create a plug-in, as long as you follow their pattern. You can even have your new plug-in call other plug-ins to make your job even easier.

Let's take a look at the skeleton of a typical plug-in:

```
imports . . .

1 class CmdLine(interfaces.plugin.PluginInterface):
      @classmethod
   2 def get_requirements(cls):
          pass

   3 def run(self):
          pass

   4 def generator(self, procs):
          pass
```

The main steps here are to create your new class to inherit from the `PluginInterface`1, define your plug-in's requirements 2, define the `run` method 3, and define the `generator` method 4. The `generator` method is optional, but separating it from the `run` method is a useful pattern you'll see in many plug-ins. By separating it and using it as a Python generator, you can get faster results and make your code easier to understand.

Let's follow this general pattern to create a custom plug-in that will check for processes that are not protected by *address space layout randomization (ASLR)*. ASLR mixes up the address space of a

vulnerable process, which affects the virtual memory location of heaps, stacks, and other operating system allocations. That means that exploit writers cannot determine how the address space of the victim process is laid out at the time of attack. Windows Vista was the first Windows release with ASLR support. In older memory images like Windows XP, you won't see ASLR protection enabled by default. Now, with recent machines (Windows 10), almost all processes are protected.

ASLR doesn't mean the attacker is out of business, but it makes the job much more complicated. As a first step in reconnoitering the processes, we'll create a plug-in to check if a process is protected by ASLR.

Let's get started. Create a directory called *plugins*. Under that directory, create a *windows* directory to contain your custom plug-ins for Windows machines. If you create plug-ins to target a Mac or Linux machine, create a directory named *mac* or *linux*, respectively.

Now, in the *plugins/windows* directory, let's write our ASLR-checking plug-in, *aslrcheck.py*:

```
# Search all processes and check for ASLR protection
#
from typing import Callable, List

from volatility.framework import constants, exceptions,
interfaces, renderers
from volatility.framework.configuration import requirements
from volatility.framework.renderers import format_hints
from volatility.framework.symbols import intermed
from volatility.framework.symbols.windows import extensions
from volatility.plugins.windows import pslist

import io
import logging
import os
import pefile

vollog = logging.getLogger(__name__)

IMAGE_DLL_CHARACTERISTICS_DYNAMIC_BASE = 0x0040
IMAGE_FILE_RELOCS_STRIPPED = 0x0001
```

We first handle the imports we'll need, plus the `pefile` library for analyzing Portable Executable (PE) files. Now let's write a helper function to do that analysis:

```
1 def check_aslr(pe):
      pe.parse_data_directories([

  pefile.DIRECTORY_ENTRY['IMAGE_DIRECTORY_ENTRY_LOAD_CONFIG']
      ])
      dynamic = False
      stripped = False

   2 if (pe.OPTIONAL_HEADER.DllCharacteristics &
            IMAGE_DLL_CHARACTERISTICS_DYNAMIC_BASE):
            dynamic = True
   3 if pe.FILE_HEADER.Characteristics &
  IMAGE_FILE_RELOCS_STRIPPED:
            stripped = True
   4 if not dynamic or (dynamic and stripped):
            aslr = False
      else:
            aslr = True
      return aslr
```

We pass a PE file object to the `check_aslr` function 1, parse it, and then check for whether it has been compiled with the DYNAMIC base setting 2 and whether the file relocation data has been stripped out 3. If it's not dynamic, or was perhaps compiled as dynamic but stripped of its relocation data, then the PE file is not protected by ASLR 4.

With the `check_aslr` helper function ready to go, let's create our `AslrCheck` class:

```
1 class AslrCheck(interfaces.plugins.PluginInterface):

      @classmethod
      def get_requirements(cls):
          return [
          2 requirements.TranslationLayerRequirement(
                  name='primary', description='Memory layer for
  the kernel',
```

```
                architectures=["Intel32", "Intel64"]),

            3 requirements.SymbolTableRequirement(
                name="nt_symbols", description="Windows
kernel symbols"),

        4 requirements.PluginRequirement(
                name='pslist', plugin=pslist.PsList, version=
(1, 0, 0)),

        5 requirements.ListRequirement(name = 'pid',
             element_type = int,
             description = "Process ID to include (all others
are excluded)",
             optional = True),
            ]
```

Step one of creating the plug-in is to inherit from the `PluginInterface` object 1. Next, define the requirements. You can get a good idea of what you need by reviewing other plug-ins. Every plug-in needs the memory layer, and we define that requirement first 2. Along with the memory layer, we also need the symbols tables 3. You'll find these two requirements used by almost all plug-ins.

We'll also need the `pslist` plug-in as a requirement in order to get all the processes from memory and re-create the PE file from the process 4. Then we'll pass the re-created PE file from each process and examine it for ASLR protection.

We may want to check a single process given a process ID, so we create another optional setting that lets us pass in a list of process IDs to limit checking to just those processes 5.

```
    @classmethod
    def create_pid_filter(cls, pid_list: List[int] = None) ->
Callable[[interfaces.objects.ObjectInterface], bool]:
        filter_func = lambda _: False
        pid_list = pid_list or []
        filter_list = [x for x in pid_list if x is not None]
        if filter_list:
            filter_func = lambda x: x.UniqueProcessId not in
filter_list
        return filter_func
```

To handle the optional process ID, we use a class method to create a filter function that returns `False` for every process ID in the list; that is, the question we're asking the filter function is whether to filter out a process, so we return `True` only if the PID is not in the list:

```
    def _generator(self, procs):
        pe_table_name =
intermed.IntermediateSymbolTable.create( 1
            self.context,
            self.config_path,
            "windows",
            "pe",
            class_types=extensions.pe.class_types)

        procnames = list()
        for proc in procs:
            procname = proc.ImageFileName.cast("string",
                max_length=proc.ImageFileName.vol.count,
errors='replace')
            if procname in procnames:
                continue
            procnames.append(procname)

            proc_id = "Unknown"
            try:
                proc_id = proc.UniqueProcessId
                proc_layer_name = proc.add_process_layer()
            except exceptions.InvalidAddressException as e:
                vollog.error(f"Process {proc_id}: invalid
address {e} inlayer {e.layer_name}")
                continue

           peb = self.context.object( 2
                    self.config['nt_symbols'] +
constants.BANG + "_PEB",
                    layer_name = proc_layer_name,
                    offset = proc.Peb)

            try:
                dos_header = self.context.object(
                        pe_table_name + constants.BANG +
"_IMAGE_DOS_HEADER",
                        offset=peb.ImageBaseAddress,
                        layer_name=proc_layer_name)
            except Exception as e:
```

```
                continue

            pe_data = io.BytesIO()
            for offset, data in dos_header.reconstruct():
                pe_data.seek(offset)
                pe_data.write(data)
            pe_data_raw = pe_data.getvalue() 3
            pe_data.close()

            try:
                pe = pefile.PE(data=pe_data_raw) 4
            except Exception as e:
                continue

            aslr = check_aslr(pe) 5

            yield (0, (proc_id, 6
                       procname,

format_hints.Hex(pe.OPTIONAL_HEADER.ImageBase),
                       aslr,
                       ))
```

We create a special data structure called `pe_table_name`1 to use as we loop over each process in memory. Then we get the Process Environment Block (PEB) memory region associated with each process and put it into an object2. The *PEB* is a data structure for the current process that contains a wealth of information on the process. We write that region into a file-like object (`pe_data`) 3, create a PE object using the `pefile` library 4, and pass it to our `check_aslr` helper method 5. Finally, we yield the tuple of information containing the process ID, process name, memory address of the process, and a Boolean result from the ASLR protection check 6.

Now we create the `run` method, which needs no arguments since all settings are populated in the config object:

```
    def run(self):
     1 procs = pslist.PsList.list_processes(self.context,

self.config["primary"],
```

```
self.config["nt_symbols"],
                                                    filter_func =
                self.create_pid_filter(self.config.get('pid',
None)))
     2 return renderers.TreeGrid([
            ("PID", int),
            ("Filename", str),
            ("Base", format_hints.Hex),
            ("ASLR", bool)],
            self._generator(procs))
```

We get the list of processes using the `pslist` plug-in 1 and return the data from the generator using the `TreeGrid` renderer 2. The `TreeGrid` renderer is used by many plug-ins. It ensures that we get one line of results for each process analyzed.

Kicking the Tires

Let's take a look at one of the images made available at the Volatility site: Malware - Cridex. For your custom plug-in, provide the `-p` switch with the path to your *plugins* folder:

```
PS>vol -p .\plugins\windows -f cridex.vmem
aslrcheck.AslrCheck
Volatility 3 Framework 1.2.0-beta.1
Progress:    0.00               Scanning primary2 using
PdbSignatureScanner
PID     Filename        Base    ASLR

368     smss.exe        0x48580000      False
584     csrss.exe       0x4a680000      False
608     winlogon.exe    0x1000000       False
652     services.exe    0x1000000       False
664     lsass.exe       0x1000000       False
824     svchost.exe     0x1000000       False
1484    explorer.exe    0x1000000       False
1512    spoolsv.exe     0x1000000       False
1640    reader_sl.exe   0x400000        False
788     alg.exe         0x1000000       False
1136    wuauclt.exe     0x400000        False
```

As you can see, this is a Windows XP machine, and there are no ASLR protections on any process.

Next is the result for a clean, up-to-date Windows 10 machine:

```
PS>vol -p .\plugins\windows -f WinDev2007Eval-Snapshot4.vmem
aslrcheck.AslrCheck
Volatility 3 Framework 1.2.0-beta.1
Progress:   33.01               Scanning primary2 using
PdbSignatureScanner
PID     Filename        Base    ASLR

316     smss.exe        0x7ff668020000  True
428     csrss.exe       0x7ff796c00000  True
500     wininit.exe     0x7ff7d9bc0000  True
568     winlogon.exe    0x7ff6d7e50000  True
592     services.exe    0x7ff76d450000  True
600     lsass.exe       0x7ff6f8320000  True
696     fontdrvhost.ex  0x7ff65ce30000  True
728     svchost.exe     0x7ff78eed0000  True

Volatility was unable to read a requested page:
Page error 0x7ff65f4d0000 in layer primary2_Process928 (Page
Fault at entry 0xd40c9d88c8a00400 in page entry)

 * Memory smear during acquisition (try re-acquiring if
possible)
 * An intentionally invalid page lookup (operating system
protection)
 * A bug in the plugin/volatility (re-run with -vvv and file
a bug)

No further results will be produced
```

Not too much to see here. Every listed process is protected by ASLR. However, we also see a memory smear. A *memory smear* occurs when the contents of the memory changes as the memory image is taken. That results in the memory table descriptions not matching the memory itself; alternatively, the virtual memory pointers may reference invalid data. Hacking is hard. As the error description says, you can try reacquiring the image (finding or creating a new snapshot).

Let’s check the PassMark Windows 10 sample memory image:

```
PS>vol -p .\plugins\windows -f WinDump.mem
aslrcheck.AslrCheck
```

```
Volatility 3 Framework 1.2.0-beta.1
Progress:    0.00               Scanning primary2 using
PdbSignatureScanner
PID     Filename        Base    ASLR

356     smss.exe        0x7ff6abfc0000  True
2688    MsMpEng.exe     0x7ff799490000  True
2800    SecurityHealth  0x7ff6ef1e0000  True
5932    GoogleCrashHan  0xed0000        True
5380    SearchIndexer.  0x7ff6756e0000  True
3376    winlogon.exe    0x7ff65ec50000  True
6976    dwm.exe         0x7ff6ddc80000  True
9336    atieclxx.exe    0x7ff7bbc30000  True
9932    remsh.exe       0x7ff736d40000  True
2192    SynTPEnh.exe    0x140000000     False
7688    explorer.exe    0x7ff7e7050000  True
7736    SynTPHelper.ex  0x7ff7782e0000  True
```

Nearly all processes are protected. Only the single process `SynTPEnh.exe` isn't ASLR protected. An online search shows that this is a software component of Synaptics Pointing Device, probably for touch screens. As long as that process is installed in *c:\Program Files*, it's probably okay, but it may be worth fuzzing later on.

In this chapter, you saw that you can leverage the power of the Volatility framework to find more information about a user's behavior and connections as well as to analyze data on any process running memory. You can use that information to better understand the target user and machine as well as to understand the mindset of a defender.

Onward!

You should have noticed by now that Python is a great language for hacking, especially when you consider the many libraries and Python-based frameworks you have available. While hackers have a plethora of tools, there's really no substitute for coding your own tools, because this gives you a deeper understanding of what those other tools are doing.

Go ahead and quickly code up a custom tool for your special requirements. Whether it's an SSH client for Windows, a web

scraper, or a command-and-control system, Python has you covered.